A3-PS022-TC368

eBPF/XDP Rate Limiting for DNS Query Floods in a Lab

Anant
anantr2006@gmail.com

AIORI Hackathon 3

October 13, 2026

Problem Description

DNS query floods exhaust the kernel before the packet ever reaches user space.

- **The problem:** high-volume UDP DNS query floods saturate Linux servers because the kernel spends most of its CPU on allocating socket buffers (`sk_buff`) for every packet, even packets that will later be dropped.
- **The impact:** softirq CPU exhaustion and full socket receive queues make the DNS service slow or unreachable, so **legitimate queries are lost** along with the flood.
- **Why current filters fall short:** netfilter/nftables rules run *after* the kernel has already allocated an `sk_buff`, so each dropped packet still costs CPU.
- **The idea:** eXpress Data Path (XDP) runs eBPF bytecode in the network driver, so packets can be dropped *before* kernel memory allocation.
- **Our challenge:** build an XDP filter with **bounded UDP/DNS parsing** plus **per-source and global rate limits**, and measure its impact on legitimate queries during three controlled, synthetic flood patterns, against no-filter and nftables baselines.
- **Scope (by design):** one Linux host pair in a closed lab; no public flood testing, no full DNS semantic parsing, no universal PRSD detection, and no claim of eliminating reflection attacks.

Background

What is needed to understand the problem and the proposed optimization.

- **DNS over UDP** is connectionless and request-sized packets are tiny, so floods are cheap to generate and source addresses are easy to forge [2].
- **Linux receive path:** NIC > driver > `sk_buff` allocation > netfilter > UDP socket queue > resolver process. The cost per packet grows at every stage, so dropping early is cheaper.
- **XDP** attaches an eBPF program at the earliest point in the driver and returns a verdict: `XDP_PASS`, `XDP_DROP`, `XDP_TX`, `XDP_REDIRECT` or `XDP_ABORTED` [1]. Modes: native (driver), generic (works on any NIC, slower) and offload.
- **eBPF verifier:** every packet read must be bounds-checked and loops must be bounded, which is why parsing has to be *bounded* and deliberately shallow.
- **eBPF maps** hold state across packets: `LRU_HASH` (per-source counters with automatic eviction) and `PERCPU_ARRAY` (lock-free per-CPU counters and buckets) [5].
- **Rate limiting** uses a *token bucket*: tokens refill at rate R up to a burst B; a query is allowed only if a token is available.
- **Baselines:** (B0) no filter and (B1) nftables rate limit on UDP port 53, both tested under identical conditions.

Literature & RFC Review

Only the most relevant studies and standards.

Source	Key point	Relevance to this project
XDP paper (CoNEXT 2018) [1]	Describes XDP as a programmable in-kernel fast path that processes packets in the driver before `sk_buff` allocation.	Basis for expecting lower CPU cost per dropped packet than netfilter.
RFC 1035 [2]	Defines the DNS message format: fixed 12-byte header (flags, QDCOUNT) followed by the question section.	Defines the minimal fields our bounded parser validates (length, QR bit, QDCOUNT).
RFC 5358 (BCP 140) [3]	Recommends limiting recursive name-server access to intended clients to avoid reflector abuse.	Rate limiting is complementary to access control; we do not claim it removes reflection attacks.
RFC 2827 (BCP 38) [4]	Ingress filtering to reject packets with spoofed source addresses.	Spoofed or many-source floods weaken per-source limits, which motivates the added global limit.
Linux kernel BPF docs [5]	Map types, verifier rules, helpers and XDP attach modes.	Implementation reference for maps, timers (`bpf_ktime_get_ns`) and generic-mode fallback.

Gap addressed: a small, repeatable lab comparison of XDP vs nftables vs no filter that reports CPU, drops *and* legitimate-query success under identical load.

Proposed Solution

An XDP program that parses only what it needs and rate-limits per source and globally.

XDP program (`xdp_dns_rl.c`)

1. **Bounded parse:** Ethernet > IPv4 > UDP with a bounds check before every read. Anything that is not IPv4/UDP dst-port 53 is passed untouched.
2. **DNS header check only:** length of at least 12 bytes and QR=0 (query); malformed packets are dropped. No question-name parsing.
3. **Per-source token bucket:** `LRU_HASH` keyed by source IP, value {tokens, last_ts}, refilled from `bpf_ktime_get_ns()`.
4. **Global token bucket:** per-CPU buckets (global rate divided across CPUs) to avoid lock contention.
5. **Verdict:** `XDP_PASS` if both buckets allow, else `XDP_DROP`; pass/drop counters by reason in a `PERCPU_ARRAY`.

Lab and test harness

- **Host pair:** a traffic-generator host and a DNS server host (e.g., Unbound). Native XDP on a capable NIC; otherwise generic mode, documented.
- **P1 single-source flood:** one source at a constant high query rate.
- **P2 many-source flood:** hundreds of synthetic source addresses, each below the per-source limit.
- **P3 bursty flood:** on/off pulses with randomized query names.
- **Always on:** a legitimate client at a fixed query rate, so success and latency can be measured.
- **Configurations:** B0 no filter, B1 nftables rate limit, X1 XDP, same limits and hardware.
- **Metrics:** queries/s, per-core CPU, drops, legitimate success % and latency; repeated runs, mean and spread.

Optimization Proposed

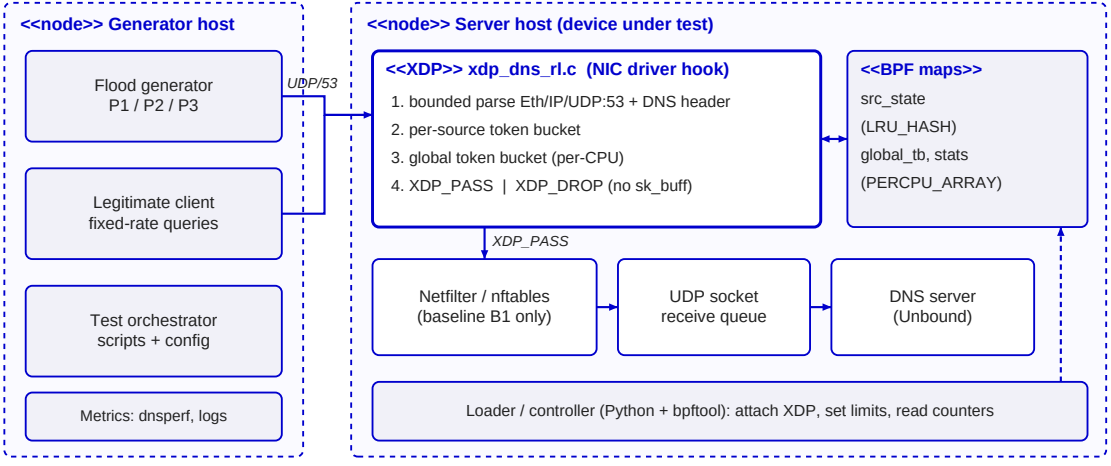
Before (no filter / nftables) versus after (XDP) at the design level.

Aspect	Before: no filter / nftables	After: XDP rate limiter
Where packets are dropped	After `sk_buff` allocation and netfilter traversal	In the driver, before `sk_buff` allocation
CPU cost per dropped packet	Allocation plus rule traversal on every packet	One short eBPF run; no allocation
Parsing	Generic match on UDP port 53	Bounded Eth/IP/UDP parse plus 12-byte DNS header check
Per-source state	Meter/set kept in netfilter	`LRU_HASH` map with automatic eviction keeps memory bounded under many-source floods
Global limit	Single shared limit	Per-CPU token buckets avoid cross-CPU contention
Fast path ordering	Rules evaluated in sequence	Cheapest checks first: non-DNS traffic is passed immediately
Tuning	Reload ruleset	Update rates in maps at runtime, no program reload
Legitimate queries	Compete with flood for CPU and socket buffer	Expected higher success rate (to be measured)

All improvements are hypotheses to be confirmed by the benchmark; results will be reported as measured.

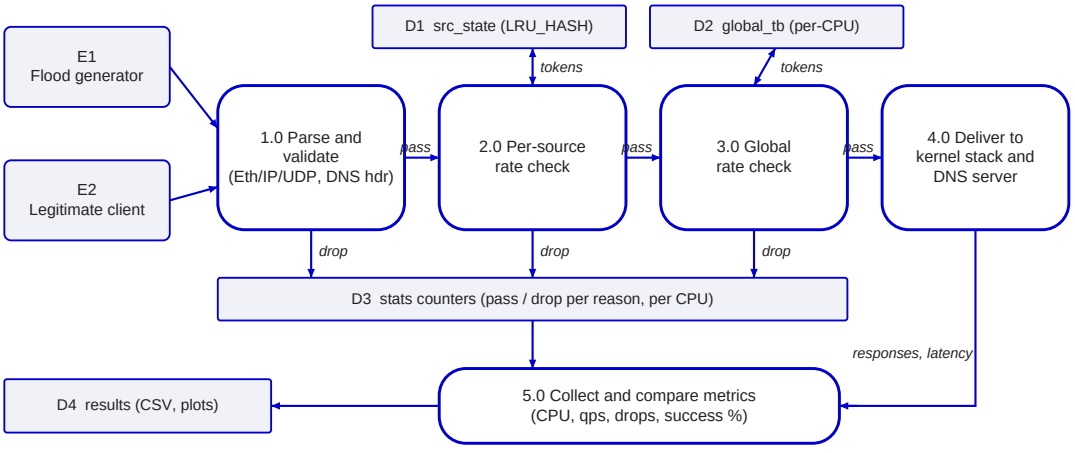
Solution Architecture & Design

Visual: component view of the lab, the XDP program and its control plane.



Data Flow Diagram

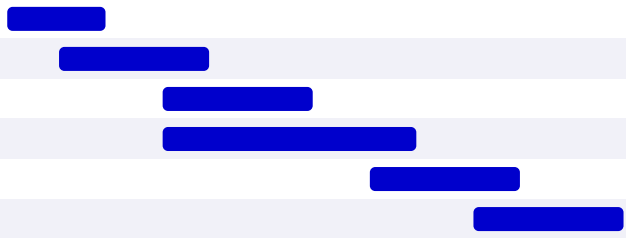
Overview of the architecture: packet path, state and measurement flow.



Deployment Timeline

Milestones and methodology: iterative build, identical-conditions benchmarking.

Milestone	Stage 1	Stage 2	Stage 3	Stage 4	Stage 5	Stage 6
M1 Lab setup + baselines B0 / B1						
M2 XDP bounded parser + counters						
M3 Per-source + global token buckets						
M4 Traffic harness: P1, P2, P3 + legit client						
M5 Benchmarks (identical conditions) + analysis						
M6 Report, repo, demo + pseudocode						



Methodology: build and test each milestone before moving on; all three configurations (B0, B1, X1) run on the same hardware, kernel, limits and traffic profiles; every scenario repeated several times and reported as mean and spread; all code, configs and logs versioned in the team repository.

Submission checklist: private GitHub repository named after the team; add **aiori-hackathon** as collaborator and, once accepted, transfer ownership; upload this PDF to the repository; prepare the prototype demonstration with a pseudocode snippet for the mentor.

Bibliography

[1] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller. “The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel.” In: *Proc. ACM CoNEXT* (2018), pp. 54–66.

[2] P. Mockapetris. “Domain Names - Implementation and Specification.” *RFC 1035*, IETF (1987).

[3] J. Damas, F. Neves. “Preventing Use of Recursive Nameservers in Reflector Attacks.” *RFC 5358 (BCP 140)*, IETF (2008).

[4] P. Ferguson, D. Senie. “Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing.” *RFC 2827 (BCP 38)*, IETF (2000).

[5] The Linux Kernel documentation. “BPF Documentation.” https://docs.kernel.org/bpf/

Thank You for Your Attention!

Questions and discussion